



Python con Pokémon

El libro del curso · Aprendé a programar desde cero
y convertite en Campeón de Kanto

Manual y libro de Python · Generado con `generar_manual.py`

Índice

0. Cómo usar este libro y el curso

- 0.1. ¿Qué es este proyecto?
- 0.2. Instalación
- 0.3. La Liga Pokémon: jugá tu progreso
- 0.4. El método: cómo trabajar cada semana

1. Tu primer programa en Python

- 1.1. `print()`: mostrar cosas en pantalla
- 1.2. El REPL: probar al instante
- 1.3. Comentarios

2. Variables y tipos de datos

- 2.1. Variables
- 2.2. Los cuatro tipos básicos
- 2.3. f-strings: armar texto con variables

3. Operadores

- 3.1. Operadores aritméticos
- 3.2. Operadores de comparación
- 3.3. Operadores lógicos

4. Cadenas de texto (strings)

- 4.1. `split` y `join`
- 4.2. Slicing: rebanar texto

5. Pedir datos: `input()`

- 5.1. Conversión de tipos

6. Condicionales: `if` / `elif` / `else`

7. Bucles: `while` y `for`

- 7.1. `while`: repetir mientras...
- 7.2. `for` y `range()`
- 7.3. `break` y `continue`

8. Funciones

- 8.1. `return`: devolver un resultado
- 8.2. Valores por defecto
- 8.3. Recursión

9. Listas y tuplas

9.1. Comprensiones de listas

9.2. Tuplas

10. Diccionarios y conjuntos

10.1. Conjuntos (sets)

10.2. Resumen de colecciones

11. Archivos y manejo de errores

11.1. try / except

12. Programación orientada a objetos (POO)

13. POO avanzado: herencia y polimorfismo

14. Módulos, pip y entornos virtuales

15. Mapa de temas del curso

15.1. Fase 1 — Linux

15.2. Fase 2 — Python básico

15.3. Descanso — Git

15.4. Fase 3 — Python avanzado

15.5. Fase 4 — Proyectos

16. Tests, preguntas frecuentes y glosario

16.1. Cómo correr los tests

16.2. Preguntas frecuentes

16.3. Glosario

0. Cómo usar este libro y el curso

¡Bienvenido! Este libro tiene **dos caras**. Por un lado es el **manual** del proyecto: te explica cómo instalarlo y cómo avanzar. Por otro lado es un **libro de Python**: a partir del capítulo 1 podés aprender el lenguaje desde cero, con ejemplos y ejercicios.

Todo está pensado para alguien que **nunca programó**. No hace falta saber nada de antes. Vamos con calma y con temática Pokémon.

0.1. ¿Qué es este proyecto?

Es un curso completo de **Linux** y **Python**, dividido en 12 semanas (más una semana de descanso de Git) y 4 proyectos finales. Cada concepto se explica con una analogía Pokémon, cada ejercicio usa Pokémon, y cada proyecto es algo que un Entrenador usaría de verdad.

0.2. Instalación

Abrí una terminal dentro de la carpeta del proyecto y corré:

```
bash setup.sh
```

Eso crea un *entorno virtual* e instala lo necesario. Después, cada vez que trabajes, actívalo:

```
source venv/bin/activate
```

CUIDADO Si `setup.sh` falla, ábrilo: está comentado línea por línea. Las dos primeras semanas son de Linux y no necesitan Python para los ejercicios de terminal.

0.3. La Liga Pokémon: jugá tu progreso

La mejor forma de hacer el curso. En vez de "hacer la tarea", subís de nivel. Abrila con:

```
python aventura.py
```

Desde ese centro de mando podés:

- Ver tu **Tarjeta de Entrenador** (nivel, EXP, rango).
- **Entrenar** una semana: corre *tus* ejercicios y te da EXP por cada test que pasás.
- Ganar las **8 medallas de gimnasio** y desbloquear logros.
- Mantener tu **racha** diaria y ver el mapa de la región.

Cuando consigas las 8 medallas, **¡sos Campeón de Kanto!**

0.4. El método: cómo trabajar cada semana

1. **Leé** la teoría de la semana (también podés leer este libro).
2. **Resolvé** los ejercicios (escribí tu código donde dice `# TU CÓDIGO ACÁ`).
3. **Probá** con los tests (ver capítulo 15).
4. **Jugá** el `interactivo.py` de la semana.
5. **Entrená** esa semana en la Liga para ganar EXP.

TIP **Regla de oro:** equivocarse es parte del juego. Escribí el código vos mismo, no copies y pegues: tipear te enseña.

1. Tu primer programa en Python

Python es un lenguaje de programación: la forma en que le damos órdenes a la computadora. Es famoso porque se lee casi como inglés y es ideal para aprender.

ANALOGÍA POKÉMON Pensá en Python como el **idioma de los Entrenadores**. Vos escribís instrucciones (el código) y la computadora las obedece, como un Pokémon bien entrenado.

1.1. print(): mostrar cosas en pantalla

La instrucción `print()` muestra texto en la pantalla. Es lo primero que hace todo programador:

```
print(";Hola, mundo Pokémon!")
print("Elegí a Pikachu como inicial")
```

Podés mostrar varias cosas separándolas con comas:

```
print("Mi Pokémon es", "Charizard")  # Mi Pokémon es Charizard
```

1.2. El REPL: probar al instante

Si escribís `python3` en la terminal, se abre el **REPL**: un modo interactivo donde probás cosas y ves el resultado al toque. El símbolo `>>>` es su prompt.

```
>>> 2 + 2
4
>>> print("Hola")
Hola
```

Para salir del REPL escribís `exit()`. Para programas de verdad, escribís el código en un archivo `.py` y lo corrés con `python3 archivo.py`.

1.3. Comentarios

Un **comentario** es texto que Python ignora. Sirve para explicar tu código. Empieza con `#`:

```
# Esto es un comentario, Python no lo ejecuta.
print("Hola")  # también se puede comentar al final de una línea
```

TIP Comentar bien tu código es de buen programador. Tu yo del futuro te lo agradece.

2. Variables y tipos de datos

2.1. Variables

Una **variable** guarda un dato con un nombre, para usarlo después. Es como una **Pokéball**: adentro guardás algo y le ponés una etiqueta.

```
nombre = "Pikachu"      # guardamos un texto
nivel = 25              # guardamos un número
print(nombre)           # Pikachu
print(nivel)            # 25
```

El nombre va a la izquierda del `=`, el valor a la derecha. Usá guiones bajos para separar palabras y no empieces con números:

```
pokemon_inicial = "Charmander" # bien
# 3pokemon = "no"              # mal: no puede empezar con número
```

2.2. Los cuatro tipos básicos

Tipo	Qué es	Ejemplo
<code>int</code>	Número entero	<code>nivel = 25</code>
<code>float</code>	Número con decimales	<code>peso = 6.0</code>
<code>str</code>	Texto (entre comillas)	<code>nombre = "Pikachu"</code>
<code>bool</code>	Verdadero o falso	<code>es_legendario = False</code>

Para ver el tipo de un valor, usás `type()`:

```
print(type(25))          # <class 'int'>
print(type("Pikachu"))  # <class 'str'>
print(type(6.0))         # <class 'float'>
print(type(True))        # <class 'bool'>
```

CUIDADO Los valores `bool` se escriben con mayúscula: `True` y `False`.

2.3. f-strings: armar texto con variables

Un **f-string** deja meter variables dentro de un texto poniendo una `f` antes de las comillas y las variables entre `{llaves}`:

```
nombre = "Pikachu"  
nivel = 25  
print(f"Mi {nombre} es nivel {nivel}")    # Mi Pikachu es nivel 25
```

CONCEPTO CLAVE Los f-strings son la forma recomendada de armar texto. ¡Usalos siempre!

3. Operadores

3.1. Operadores aritméticos

Operador	Hace	Ejemplo	Resultado
+	Suma	10 + 5	15
-	Resta	10 - 5	5
*	Multiplicación	10 * 5	50
/	División (con decimales)	10 / 4	2.5
//	División entera	10 // 4	2
%	Resto (módulo)	10 % 4	2
**	Potencia	2 ** 3	8

TIP El operador % (módulo) es súper útil: un número es par si `numero % 2 == 0`.

3.2. Operadores de comparación

Devuelven `True` o `False`:

Operador	Significa
==	igual a
!=	distinto de
> <	mayor / menor que
>= <=	mayor o igual / menor o igual

CUIDADO ¡Ojo! = asigna (guarda en una variable); == compara. No los confundas.

3.3. Operadores lógicos

Combinan condiciones: `and` (ambas), `or` (al menos una), `not` (invierte).

```
nivel = 30
tipo = "Fuego"
if nivel >= 25 and tipo == "Fuego":
    print("¡Charmander evoluciona a Charmeleon!")
```

4. Cadenas de texto (strings)

Un **string** es texto, siempre entre comillas. Tiene muchos **métodos** (acciones) para transformarlo. No cambian el original: devuelven uno nuevo.

```
nombre = "  Pikachu  "
print(nombre.upper())      # "  PIKACHU  "  (mayúsculas)
print(nombre.lower())      # "  pikachu  "  (minúsculas)
print(nombre.strip())      # "Pikachu"      (saca espacios de los bordes)
print("pikachu".capitalize()) # "Pikachu"
print("Pikachu".startswith("Pika")) # True
print(len("Pikachu"))      # 7  (longitud)
```

4.1. split y join

split parte un texto en una lista; **join** une una lista en un texto. Son clave para los archivos CSV:

```
linea = "Pikachu,Electrico,25"
partes = linea.split(",")      # ['Pikachu', 'Electrico', '25']

datos = ["Charizard", "Fuego", "50"]
linea = ",".join(datos)        # "Charizard,Fuego,50"
```

4.2. Slicing: rebanar texto

El **slicing** saca una rebanada con **[inicio:fin]**. El **fin** NO se incluye:

```
texto = "Pikachu"
#      0123456
print(texto[0:4])    # "Pika"
print(texto[4:])     # "chu"
print(texto[-3:])    # "chu"  (los últimos 3)
print(texto[::-1])   # "uhcakiP"  (al revés)
```

5. Pedir datos: input()

`input()` pausa el programa y espera que el usuario escriba algo y apriete Enter:

```
nombre = input("¿Cómo se llama tu Pokémon? ")  
print(f"¡Hola, {nombre}!")
```

CUIDADO MUY IMPORTANTE: `input()` SIEMPRE devuelve texto (str), aunque el usuario escriba un número. Si querés un número, hay que convertirlo.

5.1. Conversión de tipos

```
nivel_texto = input("¿Nivel? ")    # esto es str, ej "25"  
nivel = int(nivel_texto)          # ahora es int: 25  
print(nivel + 5)                  # 30  
  
# Forma compacta, muy común:  
edad = int(input("¿Tu edad? "))
```

Funciones de conversión: `int("25")`, `float("6.5")`, `str(25)`.

6. Condicionales: if / elif / else

`if` ejecuta un bloque solo si una condición es verdadera. `elif` agrega más condiciones, y `else` es el "si no".

```
hp = 40
if hp > 70:
    print("Tu Pokémon está sano ")
elif hp > 30:
    print("Tu Pokémon está cansado ")
elif hp > 0:
    print("¡Está grave! ")
else:
    print("Se debilitó ")
```

CONCEPTO CLAVE **La indentación importa.** El código "adentro" del `if` va con 4 espacios de sangría. Python usa esa sangría para saber qué está adentro y qué afuera. No es decorativo: es obligatorio.

Python revisa las condiciones de arriba hacia abajo y entra en la **primera** verdadera; las demás las ignora.

7. Bucles: while y for

7.1. while: repetir mientras...

while repite un bloque mientras una condición sea verdadera:

```
hp_rival = 100
while hp_rival > 0:
    print(f"Atacás. HP del rival: {hp_rival}")
    hp_rival = hp_rival - 20

print("¡Ganaste la batalla! ")
```

CUIDADO ¡Cuidado con los bucles infinitos! Si la condición nunca se vuelve falsa, el programa no termina nunca. Asegurate de que algo cambie adentro (acá, el HP baja).

7.2. for y range()

for recorre una secuencia de valores. **range()** genera números, ideal para repetir N veces:

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(f"Pokéball número {i + 1}")

for nivel in range(1, 11):  # del 1 al 10 (el 11 NO se incluye)
    print(nivel)

equipo = ["Pikachu", "Charizard"]
for pokemon in equipo:      # recorre una lista
    print(f"Tengo a {pokemon}")
```

NOTA Recordá: range(5) da 0, 1, 2, 3, 4 (empieza en 0, termina antes del 5).

7.3. break y continue

break corta el bucle por completo; **continue** saltea el resto de esta vuelta y pasa a la siguiente:

```
for pokemon in equipo:
    if pokemon == "Mewtwo":
        print("¡Lo encontramos!")
        break          # salimos del for
```

8. Funciones

Una **función** es un bloque de código con nombre que escribís una vez y usás muchas. Es como un ataque que tu Pokémon aprende una vez y usa mil veces.

```
def saludar(nombre):
    print(f"¡Hola, {nombre}!")

saludar("Ash")      # ¡Hola, Ash!
saludar("Misty")    # ¡Hola, Misty!
```

8.1. return: devolver un resultado

return hace que la función devuelva un valor para usarlo después:

```
def calcular_dano(ataque, defensa):
    return ataque - defensa

resultado = calcular_dano(50, 20)
print(resultado)    # 30
```

CONCEPTO CLAVE **print vs return:** print solo MUESTRA; return DEVUELVE un valor que podés seguir usando. Si querés usar el resultado después, usá return.

8.2. Valores por defecto

```
def atacar(nombre, dano=10):    # dano vale 10 si no se especifica
    print(f"{nombre} hizo {dano} de daño")

atacar("Pikachu")              # Pikachu hizo 10 de daño
atacar("Charizard", 35)        # Charizard hizo 35 de daño
```

8.3. Recursión

Una función puede llamarse a sí misma. Siempre necesita un **caso base** que la frene:

```
def factorial(n):
    if n <= 1:                # caso base
        return 1
    return n * factorial(n - 1) # caso recursivo

print(factorial(5))          # 120
```

9. Listas y tuplas

Una **lista** es una colección ordenada y modificable. Se escribe entre corchetes. Es como tu equipo Pokémon:

```
equipo = ["Pikachu", "Charizard", "Snorlax"]
print(equipo[0])    # Pikachu (¡los índices empiezan en 0!)
print(equipo[-1])   # Snorlax (el último)

equipo.append("Mewtwo")    # agrega al final
equipo.remove("Charizard") # borra por valor
print(len(equipo))         # cantidad de elementos
print("Pikachu" in equipo) # ¿está? -> True
```

9.1. Comprensiones de listas

Arman una lista nueva en una sola línea. Muy **pythonico**:

```
niveles = [25, 50, 40, 15]
dobles = [n * 2 for n in niveles]    # [50, 100, 80, 30]
altos = [n for n in niveles if n >= 40] # [50, 40] (con filtro)
```

9.2. Tuplas

Una **tupla** es como una lista pero **inmutable** (no se puede cambiar). Se usa para datos fijos. Se escribe con paréntesis:

```
coordenada = (10, 20)
print(coordenada[0])    # 10
# coordenada[0] = 5     # ERROR: una tupla no se modifica
```

10. Diccionarios y conjuntos

Un **diccionario** guarda pares **clave: valor**. Es como una Pokédex: buscás por nombre y obtenés los datos:

```
pikachu = {
    "nombre": "Pikachu",
    "tipo": "Electrico",
    "nivel": 25,
}
print(pikachu["nombre"])      # Pikachu
pikachu["nivel"] = 26         # cambiar un valor
pikachu["ataque"] = 55        # agregar una clave nueva
print(pikachu.get("defensa", 0)) # .get devuelve un default si no está
```

Para recorrerlo, usá `.items()`:

```
for clave, valor in pikachu.items():
    print(f"{clave}: {valor}")
```

10.1. Conjuntos (sets)

Un **set** guarda elementos únicos, sin repetidos. Ideal para sacar duplicados:

```
capturados = ["Pikachu", "Pidgey", "Pikachu", "Rattata"]
unicos = set(capturados)
print(len(unicos))    # 3 (sin contar el Pikachu repetido)
```

10.2. Resumen de colecciones

Colección	Símbolo	Característica
Lista	[]	Ordenada, modificable
Tupla	()	Ordenada, inmutable
Set	{ }	Única, sin orden
Diccionario	{clave: valor}	Pares clave-valor

11. Archivos y manejo de errores

Para que tus datos sobrevivan al cerrar el programa, los guardás en **archivos**. La forma recomendada es con `with`, que cierra el archivo solo:

```
# Escribir (¡el modo "w" pisa lo que había!)
with open("equipo.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Pikachu\n")      # \n = salto de línea
    archivo.write("Charizard\n")

# Leer línea por línea
with open("equipo.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip())      # strip saca el \n del final
```

NOTA Modos de `open()`: "r" leer, "w" escribir (reemplaza), "a" agregar al final.

11.1. try / except

Un error puede romper tu programa. Con `try / except` lo atrapas y seguís adelante:

```
try:
    numero = int(input("Nivel: "))
    print(f"El nivel es {numero}")
except ValueError:
    print("Eso no es un número válido")
```

TIP Atrapá errores específicos (como `ValueError` o `FileNotFoundError`), no un `except:` pelado.

12. Programación orientada a objetos (POO)

La POO te deja crear tus propios "moldes". Una **clase** es el molde (la especie Pikachu); un **objeto** es un individuo hecho con ese molde (tu Pikachu concreto).

```
class Pokemon:
    def __init__(self, nombre, tipo, nivel):
        # self es el propio objeto. Guardamos sus datos (atributos).
        self.nombre = nombre
        self.tipo = tipo
        self.nivel = nivel
        self.hp = 100

    def atacar(self):
        # un método (una acción)
        return f"{self.nombre} ataca con tipo {self.tipo}!"

    def __str__(self):
        # texto lindo para print()
        return f"{self.nombre} (Nivel {self.nivel})"

pikachu = Pokemon("Pikachu", "Electrico", 25)
print(pikachu.atacar()) # Pikachu ataca con tipo Electrico!
print(pikachu)          # Pikachu (Nivel 25)
```

CONCEPTO CLAVE `__init__` es el **constructor**: se ejecuta automáticamente al crear el objeto. `self` es el propio objeto y va como primer parámetro de los métodos (pero no lo pasás al llamarlos).

13. POO avanzado: herencia y polimorfismo

La **herencia** deja que una clase hija reciba todo lo de una clase padre, y agregue o cambie lo suyo:

```
class Pokemon:
    def __init__(self, nombre):
        self.nombre = nombre
    def atacar(self):
        return f"{self.nombre} usa un ataque normal"

class PokemonFuego(Pokemon):      # hereda de Pokemon
    def atacar(self):              # SOBRESCRIBE el método (polimorfismo)
        return f"{self.nombre} usa Lanzallamas! "
```

```
equipo = [PokemonFuego("Charizard"), Pokemon("Rattata")]
for p in equipo:
    print(p.atacar())    # cada uno responde a su manera
```

CONCEPTO CLAVE Polimorfismo = "muchas formas": el mismo método (atacar) se comporta distinto según la clase. Así evitás llenar el código de `if tipo == ....`

También existen `super()` (llamar al padre), `@property` (usar un método como atributo) y las clases abstractas con el módulo `abc`. Lo ves en detalle en la semana 9 del curso.

14. Módulos, pip y entornos virtuales

Un **módulo** es código ya hecho que importás. Python trae muchos de fábrica (la librería estándar):

```
import math
import random

print(math.sqrt(16))           # 4.0
print(random.choice(["Pikachu", "Onix"])) # uno al azar
```

También podés crear tus propios módulos (cualquier archivo `.py`) e importarlos, e instalar librerías externas con **pip**:

```
pip install requests
```

NOTA Un **entorno virtual** (venv) es una cajita aislada con las librerías de tu proyecto, separada del Python del sistema. El `setup.sh` del curso lo crea por vos.

Y con `requests` podés traer datos reales de internet, como de la PokéAPI:

```
import requests
datos = requests.get("https://pokeapi.co/api/v2/pokemon/pikachu").json()
print(datos["name"]) # pikachu
```

15. Mapa de temas del curso

Estos son **todos los temas** del curso, semana por semana. Cada semana tiene su teoría, ejercicios, soluciones, tests y un programa interactivo.

15.1. Fase 1 — Linux

Semana	Temas
01 — Fundamentos	Terminal, navegación, <code>ls cd pwd mkdir rm cp mv cat echo</code> , rutas, permisos
02 — Intermedio	<code>nano</code> , variables, scripts bash, <code>> >> </code> , <code>grep find ps kill chmod apt</code> , SSH

15.2. Fase 2 — Python básico

Semana	Temas
03 — Introducción	Variables, tipos, <code>print</code> , <code>input</code> , f-strings
04 — Control de flujo	<code>if/elif/else</code> , comparadores, lógicos, <code>while/for/range</code> , <code>break/continue</code>
05 — Funciones	<code>def</code> , parámetros, <code>return</code> , scope, <code>lambda</code> , recursión
06 — Listas y colecciones	Listas, tuplas, sets, diccionarios, comprensiones, <code>enumerate</code> , <code>zip</code>
07 — Cadenas y archivos	Métodos de string, slicing, <code>open</code> , CSV, <code>try/except</code>

15.3. Descanso — Git

Git: Control de Versiones — `init`, `add`, `commit`, `log`, ramas (`branch/switch/merge`), GitHub, `push/pull/clone`, `.gitignore`. Pensada para hacerse después de la semana 7.

15.4. Fase 3 — Python avanzado

Semana	Temas
08 — POO introducción	Clases, <code>__init__</code> , atributos, métodos, <code>self</code> , <code>__str__</code> , <code>__repr__</code>
09 — POO avanzado	Herencia, <code>super()</code> , polimorfismo, <code>@property</code> , clases abstractas (<code>abc</code>)
10 — Módulos y pip	<code>import</code> , módulos estándar, módulos propios, <code>pip</code> , <code>venv</code> , <code>requests</code>

15.5. Fase 4 — Proyectos

- **Semana 11 — Agenda del Entrenador:** app de consola modular con persistencia en JSON.
- **Semana 12 — Pokédex Web:** web con Flask + PokéAPI.
- **proyectos/pokedex-cli:** Pokédex de consola con sprites ASCII y favoritos.
- **proyectos/batalla-pokemon:** simulador con tipos, PP y estados alterados.
- **proyectos/agenda-entrenador** y **proyectos/pokedex-web:** versiones pulidas.

16. Tests, preguntas frecuentes y glosario

16.1. Cómo correr los tests

Los **tests** te dicen si tus ejercicios están bien. Usamos **pytest**:

```
pytest # todos los tests
pytest semana-04-python-control-de-flujo/ # una semana
pytest semana-05-python-funciones/test_ejercicios.py -v # con detalle
```

NOTA Verde = aprobado. Rojo = revisá tu solución. Por defecto los tests prueban las soluciones; la Liga los corre contra TU `ejercicios.py` para darte EXP.

16.2. Preguntas frecuentes

¿Por dónde empiezo? Por la Liga: `python aventura.py`. Después seguí la semana 01.

Me sale "command not found: python". Probá con `python3`.

Los tests me dan rojo. ¿Está mal? Significa que tu solución todavía no es correcta. Leé el error (está en español), corregí y volvé a probar. Es normal.

¿Puedo ver las soluciones? Sí, pero intentá primero vos. Mirar sin intentar es como usar un truco: ganás pero no aprendés.

16.3. Glosario

Palabra	Qué significa
Terminal	La ventana donde escribís comandos de texto.
Variable	Una "caja con nombre" donde guardás un dato.
Función	Un bloque de código con nombre que podés reusar.
Test	Un programa que revisa si tu código funciona bien.
EXP	Puntos de experiencia que ganás en la Liga al pasar tests.
Commit	Un punto de guardado de tu código en Git.
venv	Entorno virtual: cajita aislada con las librerías del curso.

*"El mejor momento para empezar a programar fue ayer. El segundo mejor momento es
ahora."
¡A atraparlos a todos!*